

Why Mining Frequent Patterns by Pattern Growth?

- ❑ Apriori: A *breadth-first search* mining algorithm
 - ❑ First find the complete set of frequent k -itemsets
 - ❑ Then derive frequent $(k+1)$ -itemset candidates
 - ❑ Scan DB again to find true frequent $(k+1)$ -itemsets
- ❑ Motivation for a different mining methodology
 - ❑ Can we develop a *depth-first search* mining algorithm?
 - ❑ For a frequent itemset p , can subsequent search be confined to only those transactions that containing p ?
- ❑ Such thinking leads to a frequent pattern growth approach: FPGrowth

FPGrowth (J. Han, J. Pei, Y. Yin, “Mining Frequent Patterns without Candidate Generation,” SIGMOD 2000)

Example: From Transactional DB to Ordered Frequent Itemlist

Example: A Sample Transactional Database

TID	Items in the Transaction
100	$\{f, a, c, d, g, i, m, p\}$
200	$\{a, b, c, f, l, m, o\}$
300	$\{b, f, h, j, o, w\}$
400	$\{b, c, k, s, p\}$
500	$\{a, f, c, e, l, p, m, n\}$

Let min_support = 3

- Scan DB once, find single item frequent pattern: $f:4, a:3, c:4, b:3, m:3, p:3$
- Sort frequent items in frequency descending order, f-list $F\text{-list} = f\text{-}c\text{-}a\text{-}b\text{-}m\text{-}p$
- Scan DB again, use the ordered frequent itemlist for each transaction to construct an FP-tree

TID	Items in the Transaction	Ordered, frequent itemlist
100	$\{f, a, c, d, g, i, m, p\}$	f, c, a, m, p
200	$\{a, b, c, f, l, m, o\}$	f, c, a, b, m
300	$\{b, f, h, j, o, w\}$	f, b
400	$\{b, c, k, s, p\}$	c, b, p
500	$\{a, f, c, e, l, p, m, n\}$	f, c, a, m, p